1. El proyecto en un vistazo

Para decir en 30 segundos. Torque organiza vehículos, trabajos y repuestos de un taller. Relaciona cada orden con el auto y el consumo de inventario, aplica permisos y conserva el historial. Presentamos una API mediante Swagger; no hay una interfaz comercial propia.

Qué está implementado

Identidad y sesiones; vehículos con dueño y baja lógica; stock y movimientos; órdenes, estados, cancelación y reposición. Roles: ADMIN y MECANICO. No incluye facturación, cobros, turnos ni avisos automáticos al cliente.

Arquitectura que todos deben poder dibujar

Cliente → gateway → auth / vehicles / inventory / workorders. Cada dominio tiene su MongoDB. Workorders consulta vehicles e inventory por HTTP. Inventory usa Redis para locks por repuesto. Son 10 contenedores: gateway, cuatro servicios, cuatro MongoDB y Redis. El monolito es histórico; Raft fue retirado.

Tecnologías y función

Node + Express: APIs. MongoDB + Mongoose: datos de negocio. Redis: coordinación temporal, no almacenamiento del stock. JWT y bcrypt: identidad y contraseñas. Docker Compose: entorno. Swagger: exploración y demo. Winston / request ID: diagnóstico.

Flujo normal de una orden

Login → vehículo activo → repuestos → orden PENDIENTE → descuentos → EN_PROCESO → FINALIZADA. Si se cancela, se intenta reponer. Crear con 201 no significa que el trabajo mecánico ya terminó. Una orden sin repuestos también es válida.

Tres fortalezas defendibles

1. Responsabilidades y persistencia separadas por dominio. MEDIDO Si se cae un servicio, solo deja de andar su dominio: con auth caído, los tokens ya emitidos siguen sirviendo.
2. JWT validado en cada servicio, roles y propiedad de órdenes.
3. Estados, movimientos y trazabilidad que permiten explicar qué ocurrió. MEDIDO En todas las cargas y caídas, el stock coincidió con la suma del libro de movimientos y nunca quedó negativo, salvo cuando forzamos el vencimiento del lock (H04).

> **Veredicto actualizado.** Es defendible como laboratorio universitario. Ahora **sí tenemos evidencia** de capacidad en una PC: ~600–800 lecturas/s, ~130 órdenes/s y ~1,8 logins/s (ver §5). Las 42 pruebas de las suites existentes pasan con datos limpios. No hay evidencia para afirmar alta disponibilidad ni preparación para producción: es una sola PC, sin réplicas.

Qué cambió respecto del repaso original

- Levantamos los 10 contenedores y corrimos **suites (42/42)**, **carga**, **caídas** y **carreras**.
- **H02, H03, H04, H05, H07 y H15 se reprodujeron en vivo.** Ya no son solo «código y dobles».
- Encontramos límites nuevos: el login es el cuello de botella, el lock rechaza en lugar de hacer cola, el listado no pagina, X24, X26 y X33 (§2).
- La sesión «se sale» a los **5 min** por defecto, y a los **15 min** en la PC de pruebas por su `.env`. Explicación en §6.

Mapa: 2 hallazgos · 3 preguntas básicas · 4 preguntas difíciles · 5 números medidos · 6 preguntas «¿y si…?» · 7 ensayo y contingencia. Mediciones del 12/11/2026 con Docker en una PC de 12 hilos; el generador de carga corría en la misma PC. Detalle: `docs/defensa/README.md` y `PLAN-RESULTADOS.md`.

2. Hallazgos que más importan

Idea central: saldo, movimientos y estado de orden no forman una única operación indivisible. Las correcciones son propuestas: no se modificó el código del producto.

H01 · Saldo y movimiento separados | Alto

Una escritura puede completarse y la otra fallar; el intento de deshacer también puede fallar. Mejora: transacción en Mongo con replica set o rediseño atómico, más reconciliación. PROBADO En el caso de control (falla el movimiento, el revert funciona), se revierte el `$inc` y el cliente recibe 500. El doble fallo no se pudo inyectar sin tocar el código.

H02 · Cancelación durante un descuento | Alto · reproducido

Creamos una orden de 3 unidades y la cancelamos a los 400 ms. Queda **CANCELADA**, el libro registra `DESCUENTO -3` y el **stock pasa de 10 a 7 sin reposición**. Reintentar la reposición da 409, así que por la API no hay forma de recuperarlo; solo un ajuste del ADMIN.

H03 · Permiso de negocio incompleto | Alto · reproducido

Un MECANICO llama a `/descontar` con un `ordenId` inventado: **200 y el stock baja de 10 a 6**. Una reposición de una orden **FINALIZADA** también da 200 y devuelve repuestos ya usados.

H04 · El lock tiene vencimiento | Alto, condicionado · reproducido

Con `LOCK_TTL_MS=300` y la demora de 1 s, las dos órdenes reciben 201 y el **stock pasa de 1 a −1**. Con el TTL normal (8 s) no ocurre: la segunda orden da 409. Mejora: guardia atómica `{stock: {$gte: n}}` en la escritura, o fencing token.

H05-H06 · Recuperación incompleta | Alto · reproducido

Si matamos workorders en medio de un descuento, la orden queda **PENDIENTE para siempre** con el stock descontado, y cancelarla no repone nada. Si inventory muere en medio de la orden, en cambio, `/reponer` la recupera (COMPLETA). Ante un timeout, el ítem queda INCIERTO y la reposición consulta el libro y devuelve bien el stock. El campo de reposición de la orden, sin embargo, queda desactualizado.

Otros límites (todos medidos)

- **H15:** enviar el mismo POST dos veces crea **dos órdenes** (el stock baja de 10 a 8). Falta una clave de idempotencia.
- **H07:** después del logout, el access sigue dando **200** hasta vencer; el refresh da 401.
- **H08 / X33:** desde el contenedor de workorders-db se lee **inventory-db sin credenciales**, y vehicles-service llega a Redis. La red interna es plana.
- **H13: mongodump/restore de inventory probado y OK** (X31). Un volumen sigue sin ser un backup.

NUEVO **Lo que el repaso original no tenía**

- **Login = cuello de botella.** bcryptjs (JS puro, costo 12) usa ~0,6 s de CPU por login y bloquea el event loop de auth. Rinde **~1,8 logins/s**. Con 50 logins simultáneos, **15 dan 503**; con 100, dan 503 **75**.
- **El lock no hace cola.** `descontar` no espera: con 100 pedidos al mismo repuesto, **1 entra y 99 reciben 409 «Recurso ocupado»**. Es correcto (nunca hay stock negativo), pero mala experiencia de uso.
- **`GET /api/ordenes` no pagina.** Con 5481 órdenes devuelve 4,4 MB en 0,9 s; con 10 usuarios a la vez, la mediana sube a **7 s**.
- **X24 / H12:** editar y dar de baja un vehículo a la vez → en **9 de 10** intentos la edición se aplica después de la baja (actualización perdida).
- **X26:** un ingreso de `2^53+1` se acepta (stock 9007199254741002, impreciso). Un cuerpo de más de 100 KB o 1 MB da **500** en lugar de 413.
- **Sin certificado HTTPS** en una instalación nueva: el HTTPS queda deshabilitado. **npm audit: 0 vulnerabilidades** en los 5 servicios y no hay secretos versionados.

Prioridad al mejorar, sin cambios: integridad local → autorización de operaciones → recuperación y concurrencia → endurecimiento. Para escalar, agregar paginación, réplicas del gateway y un bcrypt nativo o costo 10.

3. Diez respuestas esenciales

Practiquen respuestas de 20 a 30 segundos. Cada integrante debe poder explicar el control y su límite.

1. ¿Qué es Torque?

Una API de taller que relaciona vehículos, órdenes y repuestos, con permisos e historial. La demo usa Swagger; no mostramos un frontend comercial.

2. ¿Qué partes lo componen?

Un gateway y cuatro servicios: auth, vehicles, inventory y workorders. Cada dominio tiene su propio Mongo y las órdenes se coordinan por HTTP. Separar en carpetas, por sí solo, no crea microservicios: la prueba es que **con inventory caído siguen funcionando los vehículos y las órdenes sin repuestos** (medido).

3. ¿Por qué estas tecnologías?

Node/Express sirven las APIs, Mongo guarda los documentos del dominio, Redis coordina el acceso al stock y JWT identifica a los usuarios. SQL sería una alternativa válida, y tampoco eliminaría por sí solo las fallas entre servicios.

4. ¿Qué significa crear una orden?

Se validan el vehículo y los repuestos, se guarda como PENDIENTE, se descuentan los ítems y pasa a EN_PROCESO. FINALIZADA es una transición posterior, no lo que significa el 201.

5. ¿Mongo y Redis guardan lo mismo?

No. Mongo conserva usuarios, sesiones, vehículos, repuestos, movimientos y órdenes. Redis guarda locks con TTL y propietario; el stock vive en Mongo. **Con Redis caído** se siguen leyendo los stocks, pero toda escritura da 503: preferimos consistencia antes que disponibilidad (CP).

6. ¿Cómo autentican y autorizan?

Cada servicio valida firma, emisor, audiencia y vencimiento del JWT; después aplica el rol y, en órdenes, la propiedad. El historial por vehículo es visible para cualquier usuario autenticado, por diseño.

7. ¿Logout invalida todo acceso? ¿Cuánto dura la sesión?

Logout revoca la renovación. El access ya firmado sigue valiendo hasta expirar (**medido: da 200 después del logout**): **5 minutos por defecto en el código** (requisito 3), pero **lo configuramos en 12 h por `.env`** para que no moleste. Es un trade-off consciente: con 12 h, un token robado o usado después del logout vale hasta 12 h. Detalle en §6.

8. ¿Qué sobrevive a reiniciar Docker?

Los datos de Mongo en volúmenes, si se conservan esos volúmenes. **Medido: el reinicio completo tarda ~8 s, los datos quedan intactos y los tokens viejos siguen sirviendo**. Un volumen no es un backup, pero mongodump/restore está probado.

9. ¿Cómo localizan una falla?

Seguimos el `x-request-id` entre servicios y contrastamos status, logs, salud y métricas con la orden, los movimientos y el saldo. Que el gateway responda no prueba que todo el sistema esté listo.

10. ¿Qué se verificó realmente?

Además de la auditoría de código, el 12/11 corrimos sobre los 10 contenedores las **tres suites (42/42 PASS)**, carga de lecturas, logins y órdenes, nueve escenarios de caída y seis carreras. Después de cada prueba controlamos las invariantes: stock igual a la suma del libro, stock no negativo y estado de la orden coherente con sus movimientos. Una prueba de carga en una sola PC mide esta PC, no producción.

Un script que termina con código 0 no demuestra que todos los resultados de negocio sean correctos. Por eso las pruebas nuevas verifican las invariantes además del status HTTP.

4. Las cinco preguntas difíciles

11. ¿Redis garantiza que el stock nunca sea negativo?

Respuesta: el lock protege mientras mantiene la exclusión. Lo medimos: con el TTL normal, **100 pedidos simultáneos nunca dejaron stock negativo**; con stock 5 y 30 clientes reintentando, se concretaron **exactamente 5 y el stock terminó en 0**. Pero si el lock vence mientras sigue la operación, sí puede quedar negativo: **con TTL 300 ms el stock llegó a −1**. Liberar solo con el token del dueño evita borrar un lock ajeno, pero no frena al dueño viejo.
Repregunta: ¿alcanza con aumentar el TTL? Reduce la probabilidad, pero no elimina las pausas arbitrarias (GC, CPU saturada). Hace falta una guardia atómica en la base (`$gte`) o un fencing token.

12. ¿Qué pasa si cambia el stock y falla el movimiento?

Respuesta: son escrituras separadas. Hay una compensación local de mejor esfuerzo (probada: revierte y devuelve 500), pero puede fallar o no ejecutarse si el proceso se cae, y entonces el saldo y el libro divergen.
Repregunta: ¿Mongo no es atómico? Una escritura sobre un documento sí lo es; dos documentos distintos no forman una transacción. Para eso Mongo necesita un replica set: el nuestro es standalone.

13. ¿Cancelar devuelve siempre el stock?

Respuesta: no. **Lo reprodujimos:** cancelamos a los 400 ms y la orden quedó CANCELADA con 3 unidades consumidas y sin reposición. El ítem todavía estaba PENDIENTE, así que quedó fuera de la compensación.
Repregunta: ¿reiniciar lo resuelve? No. Medido: una orden huérfana después de un crash queda PENDIENTE para siempre. Falta un reconciliador durable.

14. ¿Un usuario con JWT válido puede romper el flujo?

Respuesta: sí, y **lo hicimos**: un descuento con una orden inventada da 200 y una reposición de una orden finalizada también da 200. Identidad válida no equivale a autorización para cualquier efecto.
Repregunta: ¿alcanza con ocultar la ruta en Swagger? No: HTTP la sigue aceptando. Se necesita autorización de servicio a servicio, ligada al estado de la orden.

15. ¿Timeout, sobrecarga e inconsistencia son lo mismo?

Respuesta: no. Timeout es una espera agotada; sobrecarga, demanda mayor que la capacidad; inconsistencia, una relación de datos violada. **Medido:** con 1000 usuarios leyendo hubo 0 errores, pero la mediana subió a 1,2 s (sobrecarga sin inconsistencia). Con un timeout hacia inventory el ítem quedó INCIERTO, y la reposición consultó el libro y devolvió bien el stock: el servidor puede escribir aunque el cliente deje de esperar.
Repregunta: ¿repetimos el POST? Primero hay que consultar órdenes y movimientos: no hay idempotencia en el alta (medido: el doble POST creó dos órdenes). **Ahora sí tenemos cifras**: ~600–800 lecturas/s, ~130 órdenes/s y ~1,8 logins/s (§5).

> **Frase para recordar:** «Podemos explicar qué controles implementamos, dónde termina cada garantía y **lo medimos**. Primero verificamos qué efecto se confirmó; después decidimos cómo recuperar».

5. Números medidos (12/11/2026)

Docker en una PC de 12 hilos; el generador de carga corría en la misma máquina y le quitaba CPU. Los usuarios virtuales son en lazo cerrado: cada uno pide, espera la respuesta y vuelve a pedir.

Lecturas (GET de vehículos y repuestos)

Usuarios	req/s	p50	p95	p99	Errores
1	117	7 ms	18 ms	34 ms	0
10	490	17 ms	44 ms	58 ms	0
50	774	52 ms	126 ms	184 ms	0
100	573	150 ms	344 ms	720 ms	0
250	589	379 ms	787 ms	1,4 s	0
500	606	712 ms	1,1 s	5,4 s	0
1000	630	1,2 s	4,8 s	9,3 s	0

Pasados los ~50 usuarios no rinde más: solo crece la cola (ley de Little: L = λ·W). El cuello es el **gateway**, un proceso Node al 93 % de un núcleo.

Logins simultáneos (bcrypt costo 12)

Logins	Tiempo	p50	Resultado
1	0,6 s	589 ms	1 OK
10	5,7 s	5,1 s	10 OK
25	13,4 s	9,4 s	25 OK
50	15,1 s	11,2 s	**35 OK, 15 × 503**
100	15,1 s	15,1 s	**25 OK, 75 × 503**

Rinde ~1,8 logins/s. El 503 aparece cuando la espera supera el timeout del gateway (15 s). Mientras dura la ráfaga, `/me` llega a tardar hasta 15 s: **bcrypt bloquea el event loop de auth**.

Órdenes con repuestos distintos

Usuarios	Órdenes/s (demora 1 s)	Órdenes/s (demora 0)	p50 (demora 0)
1	1,0	23	39 ms
10	9,2	59	159 ms
50	39	93	519 ms
100	63	134	663 ms

0 errores, invariantes OK. Workorders queda al 100 % de CPU. La «demora 1 s» es `DEMO_DELAY_MS`, que compose pone a propósito para que la carrera se vea en clase.

Todos sobre el mismo repuesto

Simultáneos	Demora 1 s (OK / 409)	Demora 0 (OK / 409)
10	1 / 9	4 / 6
50	1 / 49	31 / 19
100	**1 / 99**	41 / 59

El stock final siempre fue correcto. Con **stock 5 y 30 clientes reintentando**, se concretaron **5 órdenes, se rechazaron 25 y el stock terminó en 0**.

Caídas

Se cae…	Deja de andar	Sigue andando
auth	Los logins nuevos (503)	Todo lo demás con tokens ya emitidos
inventory	Repuestos; órdenes con repuestos (503, no queda orden registrada)	Vehículos y órdenes sin repuestos
redis	Toda escritura de stock (503, nada descontado)	Lecturas de stock
vehicles	Crear órdenes (503)	Historial y repuestos
workorders a mitad de una orden	**Orden huérfana PENDIENTE; cancelarla no repone (H05)**	—
inventory a mitad de una orden	Orden FALLIDA, reposición PENDIENTE	`/reponer` al volver → COMPLETA
reinicio completo	~8 s hasta healthy	Datos y tokens intactos

Un contenedor detenido tarda ~5 s en dar 503 (resolución DNS de Docker). Todas las cifras son de una sola PC: sirven para razonar cuellos de botella, no como SLA.

6. Preguntas «¿y si…?» con respuesta medida

¿Cada cuánto «se sale» el login?

Hay dos relojes:

- **Access token (JWT):** dura `JWT_EXPIRES_IN`, que es **5 min** por defecto en el código, en `.env.example`, en compose y en Swagger. Al vencer, cualquier pedido da **401**.
- **Sesión (refresh token):** vence tras **15 min sin renovarse** (`SESSION_IDLE`). Es un vencimiento deslizante: cada `/refresh` lo reinicia y no hay un máximo absoluto.

Swagger no renueva solo. Por eso, con el access en 5 min, cada 5 min hay que llamar a `POST /api/auth/refresh` con el refreshToken y volver a pulsar Authorize. Si pasan 15 min sin renovar, hay que loguearse de nuevo.

> **Lo que dejamos configurado:** `JWT_EXPIRES_IN=12h` y `SESSION_IDLE=12h` en el `.env` de la raíz (sin tocar código). Se aplica con `docker compose up -d --wait auth-service` y un login nuevo. Verificado: `expiresIn=43200`. Para volver a 5 min, se borran esas dos líneas. Instructivo: README raíz → «Cambiar cuánto dura la sesión».

Prueba real, con tiempos escalados 1:30 (access de 10 s = 5 min, inactividad de 30 s = 15 min):

Momento	Acción	Resultado
0 s	login y GET con el access	**200**
12 s	el mismo access, ya vencido	**401**
12 s	/refresh, y GET con el access nuevo	**200 / 200**
32 → 92 s	/refresh cada 20 s (uso activo durante 80 s, más que el límite de 30 s)	**200 ×4: la sesión sigue viva**
127 s	/refresh después de 35 s sin uso	**401 «Sesión no válida»**

¿Cada 5 min hay que volver a loguearse? No. El login entrega dos tokens. En una app real, el frontend llama solo a `/refresh` antes de que venza el access y el usuario no nota nada. Solo vuelve a loguearse si deja la app sin usar más que `SESSION_IDLE`, como el «sesión cerrada por inactividad» de un banco. El access es corto porque **no se puede revocar**: los servicios lo validan solo con la firma, así que un token robado o usado después del logout (H07) sirve hasta que vence. El refresh sí se revoca: con el logout, y si alguien lo reusa (medido: de 5 refresh concurrentes con el mismo token se acepta 1 y la sesión se revoca). Lo molesto era **Swagger**, que no renueva solo.

Caso	Configuración	Resultado
App real con frontend	access `5m` · inactividad `12h`	Seguro y sin molestias mientras se use
Demo por Swagger (lo que usamos)	access `12h` · inactividad `12h`	Cómodo; un token robado vale hasta 12 h

Frase: «El access es corto por seguridad y un frontend lo renovaría sin que el usuario lo note. Como la demo es por Swagger, lo alargamos por configuración, sabiendo que eso agranda la ventana de H07».

¿Y si entran 1000 usuarios?

Si ya están logueados y leen: 0 errores, pero la mediana sube a 1,2 s y el p99 a 9 s. Si todos hacen login a la vez, auth procesa ~1,8 por segundo y el resto recibe 503 a los 15 s. **Mejoras:** bcrypt nativo o costo 10, varias réplicas de auth y del gateway, y rate limit en el login.

¿Y si hay muchas órdenes acumuladas?

`GET /api/ordenes` devuelve todo: con 5481 órdenes son 4,4 MB y con 10 usuarios la mediana es de 7 s. **Mejora:** paginación con `limit/skip` o cursor, más índices.

¿Y si dos mecánicos piden el último repuesto?

Uno lo obtiene y el otro recibe 409; nunca queda stock negativo (medido con 2, 10, 50 y 100 pedidos). El que pierde no queda en espera: debe reintentar.

¿Y si mandan un número gigante o un cuerpo enorme?

El ingreso de 2^53+1 se acepta con pérdida de precisión, porque falta un tope (validar `Number.isSafeInteger` y un máximo razonable). Un cuerpo de más de 1 MB da 500 en lugar de 413. **Son fallas reales: decirlo y proponer el arreglo.**

¿Y si alguien entra a la red interna?

Puede leer las bases sin credenciales (medido: desde un contenedor se lee inventory-db). En el laboratorio la red de Docker es el único perímetro. **Mejora:** autenticación en Mongo y Redis, una red por servicio y TLS interno.

¿Por qué no 2PC? ¿Qué elegiste en CAP?

2PC bloquea si cae el coordinador y requiere participantes que lo soporten. Elegimos una saga con compensación (DESCUENTO → REPOSICION). Para el stock elegimos consistencia antes que disponibilidad (CP): si Redis cae, no se descuenta (503).

7. Ensayo y último repaso

Guion de 13 minutos

0-1: problema y alcance. 1-4: recorrido funcional en Swagger. 4-6: arquitectura. 6-8: datos, lock y permisos. 8-10: Docker, logs y un 401/403 esperado. 10-12: **números medidos** (§5), límites y mejoras prioritarias. 12-13: cierre. Una persona opera y otra observa estados y logs.

Demostración mínima que deben ensayar

1. Login y Authorize sin proyectar secretos.
2. Consultar un vehículo y un repuesto con stock suficiente. OJO **Usar BUJ-NGK-001 (stock 8)**. FIL-ACE-001 tiene stock 1 en la semilla y la segunda orden falla.
3. Crear una orden con un solo repuesto, cantidad 1, y comprobar 201 y EN_PROCESO. Tarda ~1 s por `DEMO_DELAY_MS`.
4. Mostrar el descuento, el movimiento y el historial; finalizar la orden.
5. En otra orden ya EN_PROCESO, cancelar con motivo y comprobar la reposición COMPLETA.
6. Mostrar una consulta sin token (401) o una acción sin permiso (403).

Antes de entrar a exponer

- Correr el ensayo automático en la máquina real. Debe terminar en «Todo OK»; crea 2 órdenes de prueba: `GW=http://localhost:3000 PROYECTO=torque node scripts/defensa/ensayo-demo.mjs`
- En el `.env` de esa máquina, poner `JWT_EXPIRES_IN=12h` y `SESSION_IDLE=12h`, después `docker compose up -d --wait auth-service` y loguearse de nuevo. El script imprime el valor activo. Si te preguntan: «el default es 5 min; lo alargamos a propósito y su costo es H07».
- `LAB_ENDPOINTS=false`. Docker y Compose funcionando, Swagger accesible, cuentas ADMIN y MECANICO, datos ficticios, logs y request IDs disponibles.
- **No correr pruebas de carga ni de caídas en la máquina de la demo:** los scripts están pensados para el stack descartable (`torque-defensa`, puerto 3100).

Si algo falla: observar antes de repetir

400/401/403/409: leer el cuerpo y explicar si es validación, identidad, permiso o conflicto. Un 409 «Recurso ocupado» significa que otro pedido tiene el lock: esperar 1 s y reintentar.

500/503/timeout: no mandar más POST, anotar la hora y el request ID, y revisar salud, logs, orden y movimientos. Si un contenedor está detenido, el 503 tarda ~5 s.

Reposición pendiente: `POST /api/ordenes/:id/reponer` (probado: la deja COMPLETA). No forzar estados en Mongo.

Cinco trampas para practicar juntos

Doble clic: crea dos órdenes (medido). **Logout:** el access sigue dando 200 (medido). **Cancelar durante la creación:** el stock queda consumido (medido). **Volumen:** no es un backup (mongodump probado). **Healthy:** no demuestra inicialización completa; con Redis caído, inventory reporta «degraded».

> **Cierre sugerido.** «Torque demuestra separación por dominios, permisos, estados y trazabilidad, y lo medimos: aguanta ~600–800 lecturas y ~130 órdenes por segundo en una PC, nunca deja el stock negativo con el lock vigente, y cada caída afecta solo a su dominio. Sus principales mejoras pendientes son atomicidad del inventario, autorización de operaciones internas, recuperación durable, login más liviano y paginación».

Base: auditoría del 12/11/2026 (commit 625d7213) más las pruebas en Docker del mismo día. Scripts en `scripts/defensa/`, resultados crudos en `docs/defensa/resultados/` y matriz E01–E42 / X01–X35 en `docs/defensa/PLAN-RESULTADOS.md`. Rama `defensa/pruebas-carga-2026-11-12`; el código del producto no se modificó.